

laC Confluent Cloud Application Resources Example

This Terraform repo provisions the application-layer resources required to demonstrate Confluent Cloud Cluster Linking between two Kafka clusters, a Sandbox (source) cluster and a Shared (destination) cluster, within a single Confluent Cloud environment. All API key credentials are automatically rotated on a configurable schedule and securely stored in AWS Secrets Manager for consumption by Java-based Kafka clients.

Below is the Terraform resource visualization of the infrastructure that's created:

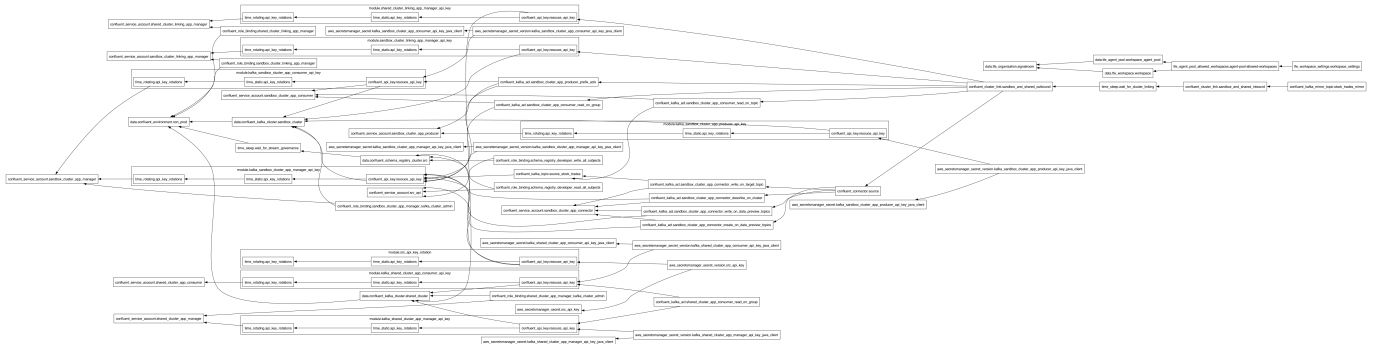


Table of Contents

- [1.0 Architecture Overview](#)
- [2.0 Why the PrivateLink Configuration Lives in a Separate Workspace](#)
 - [2.1. Different Lifecycles](#)
 - [2.2. Different Blast Radius](#)
 - [2.3. Different Permission Boundaries](#)
 - [2.4. Team Ownership Alignment](#)
 - [2.5. Dependency Ordering Without Tight Coupling](#)
- [3.0 What This Workspace Provisions](#)
- [4.0 Let's Get Started](#)
 - [4.1 Deploy the Infrastructure](#)
 - [4.1.1 Handling DNS Resolution Errors](#)
 - [4.1.2 Cluster Linking Error](#)
 - [4.2 Teardown the Infrastructure](#)
 - [4.2.1 Handling Cluster Link Deletion Error\(s\)](#)
- [5.0 Resources](#)
 - [5.1 Terminology](#)
 - [5.2 Related Documentation](#)

1.0 Architecture Overview

Below is the full topology of the infrastructure provisioned by this Terraform codebase, which is designed to demonstrate Confluent Cloud Cluster Linking and API key rotation with AWS Secrets Manager.

title: "Confluent Cloud Cluster Linking with AWS PrivateLink –

Architecture"

graph TB

%% — Styling

```

classDef confluent fill:#172554,stroke:#1e40af,color:#fff
classDef kafka fill:#1e3a5f,stroke:#3b82f6,color:#fff
classDef sa fill:#0f4c75,stroke:#6dd5ed,color:#fff
classDef topic fill:#1a535c,stroke:#4ecdc4,color:#fff
classDef connector fill:#6b21a8,stroke:#a78bfa,color:#fff
classDef aws fill:#ff9900,stroke:#cc7a00,color:#fff
classDef secret fill:#d97706,stroke:#f59e0b,color:#fff
classDef tfc fill:#7c3aed,stroke:#a78bfa,color:#fff
classDef link fill:#059669,stroke:#34d399,color:#fff
classDef sr fill:#0e7490,stroke:#22d3ee,color:#fff
classDef mirror fill:#065f46,stroke:#6ee7b7,color:#fff

```

%% — Confluent Cloud Environment

```

subgraph CC["Confluent Cloud Environment (non-prod)"]
  direction TB

```

%% — Schema Registry

```

  subgraph SRcluster["Schema Registry Cluster"]

```

```

    SR_API["src_api

```

```

Service Account"]:::sa

```

```

    SR_RB_R["DeveloperRead

```

```

all subjects"]

```

```

    SR_RB_W["DeveloperWrite

```

```

all subjects"]

```

```

    SR_API --> SR_RB_R

```

```

    SR_API --> SR_RB_W

```

```

  end

```

```

  SRcluster:::sr

```

%% — Sandbox Cluster (Source)

```

  subgraph Sandbox["Sandbox Kafka Cluster – Source"]
    direction TB

```

```

    SB_MGR["sandbox_cluster_app_manager

```

```

CloudClusterAdmin"]:::sa

```

```

    SB_PROD["sandbox_cluster_app_producer

```

```

Service Account"]:::sa

```

```

    SB_CONS["sandbox_cluster_app_consumer

```

```

Service Account"]:::sa

```

```

    SB_CONN_SA["sandbox_cluster_app_connector

```

```

Service Account"]:::sa

```

```

    SB_TOPIC["dev-stock_trades

```

```

Kafka Topic"]:::topic

```

```

    DATAGEN["DataGen Source Connector

```

```

STOCK_TRADES / AVRO"]:::connector

```

```

    SB_ACL_PROD["ACLs: READ, WRITE, DESCRIBE

```

```

on dev-stock_trades"]

```

```

        SB_ACL_CONS_T["ACL: READ
on dev-stock_trades"]
        SB_ACL_CONS_G["ACL: READ
on consumer group"]
        SB_ACL_CONN["ACLs: DESCRIBE cluster,
WRITE + CREATE topics"]

        SB_PROD --> SB_ACL_PROD --> SB_TOPIC
        SB_CONS --> SB_ACL_CONS_T --> SB_TOPIC
        SB_CONS --> SB_ACL_CONS_G
        SB_CONN_SA --> SB_ACL_CONN --> SB_TOPIC
        DATAGEN -->|"produces AVRO"| SB_TOPIC
    end
    Sandbox:::kafka

%% — Shared Cluster (Destination) —————
    subgraph Shared["Shared Kafka Cluster – Destination"]
        direction TB

        SH_MGR["shared_cluster_app_manager
CloudClusterAdmin"]:::sa
        SH_CONS["shared_cluster_app_consumer
Service Account"]:::sa

        MIRROR["dev-stock_trades
Mirror Topic"]:::mirror

        SH_ACL_CONS_G["ACL: READ
on consumer group"]

        SH_CONS --> SH_ACL_CONS_G
        SH_CONS -. ->|"consumes"| MIRROR
    end
    Shared:::kafka

%% — Cluster Linking —————
    subgraph CLinking["Bidirectional Cluster Link"]
        direction LR
        CL_SB_SA["sandbox_cluster_linking_app_manager
EnvironmentAdmin"]:::sa
        CL_SH_SA["shared_cluster_linking_app_manager
EnvironmentAdmin"]:::sa
        CL_OUT["Outbound Link
Sandbox → Shared"]:::link
        CL_IN["Inbound Link
Shared → Sandbox"]:::link
        CL_SB_SA --> CL_OUT
        CL_SH_SA --> CL_IN
    end
    end
    CC:::confluent

%% — Cluster Link connections —————
    SB_TOPIC ==>|"replicates via

```

```

cluster link"| MIRROR
  CL_OUT --> Sandbox
  CL_OUT --> Shared
  CL_IN --> Shared
  CL_IN --> Sandbox

%% — AWS Secrets Manager —————
subgraph AWS["AWS Account"]
  direction TB

  subgraph SM["AWS Secrets Manager"]
    direction TB
    SEC_SR["schema_registry_cluster
URL + basic auth"]:::secret
    SEC_SB_MGR["sandbox_cluster/
app_manager/java_client"]:::secret
    SEC_SB_CONS["sandbox_cluster/
app_consumer/java_client"]:::secret
    SEC_SB_PROD["sandbox_cluster/
app_producer/java_client"]:::secret
    SEC_SH_MGR["shared_cluster/
app_manager/java_client"]:::secret
    SEC_SH_CONS["shared_cluster/
app_consumer/java_client"]:::secret
  end
  SM:::aws
end

%% — Secret wiring —————
SR_API -->|"API key stored"| SEC_SR
SB_MGR -->|"JAAS config stored"| SEC_SB_MGR
SB_CONS -->|"JAAS config stored"| SEC_SB_CONS
SB_PROD -->|"JAAS config stored"| SEC_SB_PROD
SH_MGR -->|"JAAS config stored"| SEC_SH_MGR
SH_CONS -->|"JAAS config stored"| SEC_SH_CONS

%% — Terraform Cloud —————
subgraph TFC["Terraform Cloud"]
  direction LR
  TFC_ORG["signalroom
Organization"]:::tfc
  TFC_WS["iac-cc-app-resources-example
Workspace"]:::tfc
  TFC_AP["signalroom-iac-tfc-agents-pool
Agent Pool"]:::tfc
  TFC_ORG --- TFC_WS
  TFC_WS -->|"execution mode: agent"| TFC_AP
end

%% — API Key Rotation Module —————
ROTATION["iac-confluent-api_key_rotation-tf_module
v0.23.00.000
Rotates keys every N days,
retains M keys"]

```

```
ROTATION -. ->|"manages keys for"| SB_MGR
ROTATION -. ->|"manages keys for"| SB_PROD
ROTATION -. ->|"manages keys for"| SB_CONS
ROTATION -. ->|"manages keys for"| SH_MGR
ROTATION -. ->|"manages keys for"| SH_CONS
ROTATION -. ->|"manages keys for"| CL_SB_SA
ROTATION -. ->|"manages keys for"| CL_SH_SA
ROTATION -. ->|"manages keys for"| SR_API
```

The **Schema Registry** cluster is shared across the environment and its credentials are also stored in AWS Secrets Manager.

2.0 Why the PrivateLink Configuration Lives in a Separate Workspace

The AWS PrivateLink networking infrastructure is intentionally managed in its own Terraform Cloud workspace ([iac-cc-aws-privatelink-infrastructure-networking-example](#)), separate from this application-resources workspace ([iac-cc-app-resources-example](#)). There are several important reasons for this separation:

2.1. Different Lifecycles

Network infrastructure (VPCs, subnets, PrivateLink endpoint services, VPC endpoints, DNS hosted zones) changes infrequently, often only at initial setup or during major topology changes. Application resources such as service accounts, API keys, ACLs, topics, connectors, and cluster links change much more frequently as teams iterate on their streaming workloads. Coupling them together would force unnecessary plan/apply cycles on stable networking resources every time an application-level change is needed.

2.2. Different Blast Radius

A misconfigured Terraform apply against PrivateLink resources could sever private connectivity for every service account and application relying on the cluster. By isolating networking in its own workspace, accidental disruption from application-layer changes is impossible, and vice versa. Each workspace has its own state file, so a corrupted or rolled-back state in one workspace cannot cascade into the other.

2.3. Different Permission Boundaries

PrivateLink provisioning requires elevated AWS permissions (creating VPC endpoints, modifying route tables, managing private hosted zones) and Confluent Cloud permissions (accepting PrivateLink connections on enterprise clusters). Application resources require only Confluent Cloud service-account-level permissions and limited AWS access for Secrets Manager. Splitting the workspaces allows tighter IAM scoping, the application workspace's Terraform Cloud agent needs only `secretsmanager:*` on a narrow path, not broad VPC/EC2 permissions.

2.4. Team Ownership Alignment

In many organizations, a platform or network engineering team owns the PrivateLink setup, while application or data engineering teams own the Kafka resources layered on top. Separate workspaces map cleanly to separate code repositories, PR review cycles, and on-call responsibilities.

2.5. Dependency Ordering Without Tight Coupling

This workspace references the already-provisioned clusters by their IDs (passed in as input variables). It does not create the clusters or their PrivateLink connections, it simply consumes them as data sources. This loose coupling means the PrivateLink workspace can be applied first (and independently validated) before this workspace is ever initialized.

3.0 What This Workspace Provisions

Layer	Resources
Sandbox Kafka Cluster	Service accounts (<code>app_manager</code> , <code>app_producer</code> , <code>app_consumer</code> , <code>app_connector</code>), role bindings, ACLs, the <code>dev-stock_trades</code> topic, a DataGen Source connector, and rotating API key pairs for each service account
Shared Kafka Cluster	Service accounts (<code>app_manager</code> , <code>app_consumer</code>), role bindings, ACLs, and rotating API key pairs
Cluster Linking	A bidirectional cluster link between Sandbox and Shared, plus a mirror topic (<code>dev-stock_trades</code>) on the Shared cluster
Schema Registry	Service account, DeveloperRead/DeveloperWrite role bindings on all subjects, and a rotating API key pair
AWS Secrets Manager	Six secrets storing JAAS credentials and bootstrap server URIs for Java Kafka clients
Terraform Cloud	Workspace configuration to run on a TFC agent pool (<code>signalroom-iac-tfc-agents-pool</code>)

4.0 Let's Get Started

4.1 Deploy the Infrastructure

The `deploy.sh` script handles authentication and Terraform execution:

```
./deploy.sh create --profile=<SSO_PROFILE_NAME> \
                  --confluent-api-key=<CONFLUENT_API_KEY> \
                  --confluent-api-secret=<CONFLUENT_API_SECRET> \
                  --tfe-token=<TFE_TOKEN> \
                  --confluent-environment-id=<CONFLUENT_ENVIRONMENT_ID> \
                  --confluent-sandbox-kafka-cluster-id=
<CONFLUENT_SANDBOX_KAFKA_CLUSTER_ID> \
                  --confluent-shared-kafka-cluster-id=
<CONFLUENT_SHARED_KAFKA_CLUSTER_ID> \
                  [--day-count=<DAY_COUNT>]
```

Here's the argument table for `deploy.sh create` command:

Argument	Required	Description
<code>--profile</code>	✓	The AWS SSO profile name. Passed directly to <code>aws sso login</code> and <code>aws2-wrap</code> for authentication, and used to resolve <code>AWS_REGION</code> , <code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , and <code>AWS_SESSION_TOKEN</code> , which are then exported as <code>TF_VAR_aws_region</code> , <code>TF_VAR_aws_access_key_id</code> , <code>TF_VAR_aws_secret_access_key</code> , and <code>TF_VAR_aws_session_token</code> for Terraform, respectively.
<code>--confluent-api-key</code>	✓	Confluent Cloud API key. Exported as <code>TF_VAR_confluent_api_key</code> for Terraform.
<code>--confluent-api-secret</code>	✓	Confluent Cloud API secret. Exported as <code>TF_VAR_confluent_api_secret</code> for Terraform.
<code>--tfe-token</code>	✓	Terraform Enterprise/Cloud API token. Exported as <code>TF_VAR_tfe_token</code> — used for authenticating the TFC Agent or remote backend.
<code>--confluent-environment-id</code>	✓	Confluent Cloud environment ID where the clusters are provisioned. Exported as <code>TF_VAR_confluent_environment_id</code> for Terraform.
<code>--confluent-sandbox-kafka-cluster-id</code>	✓	Confluent Cloud Kafka cluster ID for the Sandbox (source) cluster. Exported as <code>TF_VAR_confluent_sandbox_kafka_cluster_id</code> for Terraform.
<code>--confluent-shared-kafka-cluster-id</code>	✓	Confluent Cloud Kafka cluster ID for the Shared (destination) cluster. Exported as <code>TF_VAR_confluent_shared_kafka_cluster_id</code> for Terraform.
<code>--day-count</code>	✗	API key rotation interval in days. Exported as <code>TF_VAR_day_count</code> .

All 7 arguments are required — the script exits with code 85 if any are missing.

4.1.1 Handling DNS Resolution Errors

```
Error: error creating Kafka Topic: Post "https://lkc-111qr6.us-east-1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/topics": net/http: TLS handshake timeout
```

```
|  
|   with confluent_kafka_topic.source_stock_trades,  
|   on setup-confluent-kafka-sandbox_cluster.tf line 50, in resource  
|"confluent_kafka_topic" "source_stock_trades":  
|     50: resource "confluent_kafka_topic" "source_stock_trades" {  
|  
|  
|  
  
Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
```

```

1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|
|   with
confluent_kafka_acl.sandbox_cluster_app_producer_prefix_acls["DESCRIBE"],
|   on setup-confluent-kafka-sandbox_cluster.tf line 100, in resource
"confluent_kafka_acl" "sandbox_cluster_app_producer_prefix_acls":
|   100: resource "confluent_kafka_acl"
"sandbox_cluster_app_producer_prefix_acls" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|
|   with
confluent_kafka_acl.sandbox_cluster_app_producer_prefix_acls["WRITE"],
|   on setup-confluent-kafka-sandbox_cluster.tf line 100, in resource
"confluent_kafka_acl" "sandbox_cluster_app_producer_prefix_acls":
|   100: resource "confluent_kafka_acl"
"sandbox_cluster_app_producer_prefix_acls" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|
|   with
confluent_kafka_acl.sandbox_cluster_app_producer_prefix_acls["READ"],
|   on setup-confluent-kafka-sandbox_cluster.tf line 100, in resource
"confluent_kafka_acl" "sandbox_cluster_app_producer_prefix_acls":
|   100: resource "confluent_kafka_acl"
"sandbox_cluster_app_producer_prefix_acls" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|
|   with confluent_kafka_acl.sandbox_cluster_app_consumer_read_on_group,
|   on setup-confluent-kafka-sandbox_cluster.tf line 162, in resource
"confluent_kafka_acl" "sandbox_cluster_app_consumer_read_on_group":
|   162: resource "confluent_kafka_acl"
"sandbox_cluster_app_consumer_read_on_group" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|

```



```

|   with
confluent_kafka_acl.sandbox_cluster_app_connector_describe_on_cluster,
|   on setup-confluent-kafka-sandbox_cluster.tf line 211, in resource
"confluent_kafka_acl" "sandbox_cluster_app_connector_describe_on_cluster":
|   211: resource "confluent_kafka_acl"
"sandbox_cluster_app_connector_describe_on_cluster" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|
|   with
confluent_kafka_acl.sandbox_cluster_app_connector_create_on_data_preview_t
opics,
|   on setup-confluent-kafka-sandbox_cluster.tf line 255, in resource
"confluent_kafka_acl"
"sandbox_cluster_app_connector_create_on_data_preview_topics":
|   255: resource "confluent_kafka_acl"
"sandbox_cluster_app_connector_create_on_data_preview_topics" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-111qr6.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-111qr6/acls":
net/http: TLS handshake timeout
|
|   with
confluent_kafka_acl.sandbox_cluster_app_connector_write_on_data_preview_to
pics,
|   on setup-confluent-kafka-sandbox_cluster.tf line 277, in resource
"confluent_kafka_acl"
"sandbox_cluster_app_connector_write_on_data_preview_topics":
|   277: resource "confluent_kafka_acl"
"sandbox_cluster_app_connector_write_on_data_preview_topics" {
|
|
|
| Error: error creating Kafka ACLs: Post "https://lkc-7vvj61.us-east-
1.aws.private.confluent.cloud:443/kafka/v3/clusters/lkc-7vvj61/acls":
net/http: TLS handshake timeout
|
|   with confluent_kafka_acl.shared_cluster_app_consumer_read_on_group,
|   on setup-confluent-kafka-shared_cluster.tf line 85, in resource
"confluent_kafka_acl" "shared_cluster_app_consumer_read_on_group":
|   85: resource "confluent_kafka_acl"
"shared_cluster_app_consumer_read_on_group" {
|

```

If you encounter DNS resolution errors like the ones above during the apply process, simply re-run the `./deploy.sh create` script.

4.1.2 Cluster Linking Error

```
| Error: error creating Cluster Link: 400 Bad Request: A cluster link
| already exists with the provided link name: Cluster Link
| LL0WktYVQom94jrrQTIuDg already exists.
|
|     with confluent_cluster_link.sandbox_and_shared_inbound,
|     on setup-confluent-cluster_linking.tf line 122, in resource
| "confluent_cluster_link" "sandbox_and_shared_inbound":
|   122: resource "confluent_cluster_link" "sandbox_and_shared_inbound" {
```

If you see the above error, it indicates that the previous failed attempt left the cluster link in place. To resolve, delete the existing cluster link via the Confluent CLI:

```
confluent kafka link delete bidirectional_between_sandbox_and_shared --
cluster <SANDBOX_CLUSTER_ID> --environment <ENVIRONMENT_ID> --force
```

Then re-run the `./deploy.sh create` command.

4.2 Teardown the Infrastructure

```
./deploy.sh destroy --profile=<SSO_PROFILE_NAME> \
    --confluent-api-key=<CONFLUENT_API_KEY> \
    --confluent-api-secret=<CONFLUENT_API_SECRET> \
    --tfe-token=<TFE_TOKEN> \
    --confluent-environment-id=<CONFLUENT_ENVIRONMENT_ID>
\
    --confluent-sandbox-kafka-cluster-id=
<CONFLUENT_SANDBOX_KAFKA_CLUSTER_ID> \
    --confluent-shared-kafka-cluster-id=
<CONFLUENT_SHARED_KAFKA_CLUSTER_ID>
```

Here's the argument table for `deploy.sh destroy` command:

Argument	Required	Description
<code>--profile</code>		The AWS SSO profile name. Passed directly to <code>aws sso login</code> and <code>aws2-wrap</code> for authentication, and used to resolve <code>AWS_REGION</code> , <code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , and <code>AWS_SESSION_TOKEN</code> , which are then exported as <code>TF_VAR_aws_region</code> , <code>TF_VAR_aws_access_key_id</code> , <code>TF_VAR_aws_secret_access_key</code> , and <code>TF_VAR_aws_session_token</code> for Terraform, respectively.

Argument	Required	Description
<code>--confluent-api-key</code>	✓	Confluent Cloud API key. Exported as <code>TF_VAR_confluent_api_key</code> for Terraform.
<code>--confluent-api-secret</code>	✓	Confluent Cloud API secret. Exported as <code>TF_VAR_confluent_api_secret</code> for Terraform.
<code>--tfe-token</code>	✓	Terraform Enterprise/Cloud API token. Exported as <code>TF_VAR_tfe_token</code> — used for authenticating the TFC Agent or remote backend.
<code>--confluent-environment-id</code>	✓	Confluent Cloud environment ID where the clusters are provisioned. Exported as <code>TF_VAR_confluent_environment_id</code> for Terraform.
<code>--confluent-sandbox-kafka-cluster-id</code>	✓	Confluent Cloud Kafka cluster ID for the Sandbox (source) cluster. Exported as <code>TF_VAR_confluent_sandbox_kafka_cluster_id</code> for Terraform.
<code>--confluent-shared-kafka-cluster-id</code>	✓	Confluent Cloud Kafka cluster ID for the Shared (destination) cluster. Exported as <code>TF_VAR_confluent_shared_kafka_cluster_id</code> for Terraform.

All 7 arguments are required — the script exits with code `85` if any are missing.

4.2.1 Handling Cluster Link Deletion Error(s)

If you encounter a Cluster Link deletion error during the destroy process, you may see an error message similar to the following:

```
| Error: error deleting Cluster Link "lkc-
| 7vvj61/bidirectional_between_sandbox_and_shared": 401 Unauthorized: Not
| authorized: the authenticated user didn't have the right access to the
| resource: Cluster authorization failed.
|
```

Navigate to the Terraform directory:

```
cd terraform
```

Remove the unreachable resources from the Terraform state:

```
terraform state rm 'confluent_cluster_link.sandbox_and_shared_outbound'
terraform state rm 'confluent_cluster_link.sandbox_and_shared_inbound'
```

```
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_connector_describe_on_cluster'
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_connector_write_on_target_topic'
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_connector_create_on_data_preview_
topics'
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_connector_write_on_data_preview_t
opics'
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_producer_prefix_acls["DESCRIBE"] '
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_producer_prefix_acls["READ"] '
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_producer_prefix_acls["WRITE"] '
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_consumer_read_on_topic'
terraform state rm
'confluent_kafka_acl.sandbox_cluster_app_consumer_read_on_group'
terraform state rm 'confluent_kafka_topic.source_stock_trades'
```

Navigate back to the root directory:

```
cd ..
```

Rerun the `./deploy.sh destroy` command.

5.0 Resources

5.1 Terminology

- **ACL:** Access Control List - A list of permissions attached to an object that specifies which users or system processes can access the object and what operations they can perform.
- **AWS:** Amazon Web Services - A comprehensive cloud computing platform provided by Amazon.
- **CC:** Confluent Cloud - A fully managed event streaming platform based on Apache Kafka.
- **IaC:** Infrastructure as Code - The practice of managing and provisioning computing infrastructure through machine-readable definition files.
- **JAAS:** Java Authentication and Authorization Service - A Java security framework for user authentication and authorization.
- **TFC:** Terraform Cloud - A service that provides infrastructure automation using Terraform.
- **VPC:** Virtual Private Cloud - A virtual network dedicated to your AWS account.

5.2 Related Documentation

- [Geo-replication with Cluster Linking on Confluent Cloud](#)